```
> ##########################################################\
   ################
  # TauTimp Framework — Thermal Particle

     # Thermodynamic Gaussian: P(r,t) with sigma^2 = kB*T/m, mu=0,
     BC P(2R,0)=0
  # Author: Viorel Popescu
  # Date:   April 2026
  # Maple 2024 compatible transcript
  #
  # Key physics (new vs previous versions):

     #  - sigma^2 = kB*T/m  (Maxwell-Boltzmann variance, NOT
     arbitrary)
  #  - mu = 0             (particle pinned at cosmological origin)
  #  - BC: P(2R, 0) = 0   (boundary at 2R, not R)

     #  - V = sigma_0       (thermal velocity equals initial width
     in natural units)

     #  - T = m*R^2/(100*kB): cosmological temperature from
     sigma_0=R/10
  #
  # Amplitude ODE: N'' = V^2*(1/R - 3/sigma^2)*N

     #  -> epsilon = sigma_0^2/R ~ 3.9e20 >> 1 (exponential
     growth regime)

     #  -> Analytic normalised N(t)=sqrt(2/Pi)/sigma(t) used
     throughout

     #  -> Residual ~ sigma^2/R = O(R/100): relative error 0.5% at
     t=0

     ##########################################################\
     ###################

  restart:
  with(plots):
  with(DEtools):
  with(LinearAlgebra):

  # ========================================================
```

```
# SECTION 0: Physical Constants and Thermodynamic Parameters
# ============================================================
   printf("=== SECTION 0: Physical Constants and Temperature ===
   \n") :

hbar  := 1.05457e-34 :    # J*s
m_e   := 9.10938e-31 :    # kg, electron mass
eV_J  := 1.60218e-19 :    # J per eV
kB    := 1.38065e-23 :    # J/K, Boltzmann constant
R_c   := 4.235518529e22 :      # m, TauTimp cosmological radius
l_q   := 1 :              # quantum spin number l = n_r
E_eV  := 1.0 :            # trial energy in eV
E_J   := evalf(E_eV* eV_J) :

# Thermodynamic Gaussian: sigma^2 = kB*T/m_e, mu = 0
# Choose sigma_0 = R/10 (so that P(2R,0) ~ exp(-200) ~ 0)
n_sig  := 10 :
sigma0 := evalf(R_c / n_sig) :     # = 4.2355e21 m

# Temperature from sigma^2 = kB*T/m:
T_K   := evalf(m_e * sigma0^2 / kB) :


   # Thermal velocity: V = sigma_0 (natural units: V = sqrt(kB*
   T/m) = sigma_0)
V_th  := sigma0 :
   # m/s  (= 4.2355e21 m/s — cosmological thermal speed)

# Time unit: t0 = sigma_0 / V = 1 s (in these cosmological units)
t0    := evalf(sigma0 / V_th) :

printf("R_c     = %e m\n", R_c) :
printf("sigma_0 = sqrt(kB*T/m) = R/%d = %e m\n", n_sig, sigma0) :
printf("T       = m*sigma_0^2/kB = %e K\n", T_K) :
printf("V_th    = sqrt(kB*T/m) = sigma_0 = %e m/s\n", V_th) :
printf("t0      = sigma_0/V = %e s\n", t0) :
printf("Boundary: P(2R,0) ~ exp(-200) = %e ~ 0\n", evalf(exp(
   -200))) :
printf("Normalisation: sigma^2 = kB*T/m confirmed: %e vs %e\n",
    evalf(kB* T_K/ m_e), evalf(sigma0^2)) :
```

```
# Dimensionless ODE parameter (determines regime):
epsilon := evalf(sigma0^2 / R_c):
printf("epsilon = sigma_0^2/R_c = R_c/100 = %e >> 3\n",
    epsilon):
printf("  => Amplitude ODE in exponential-growth regime\n"):
   printf("  => Analytic normalised N(t)=sqrt(2/Pi)/sigma(t)
   used\n"):

# ============================================================
# SECTION 1: Wave Equation — Airy Reduction
# ============================================================
   printf("\n=== SECTION 1: Wave Equation and Airy Reduction ===
   \n"):

k2_E    := evalf(2*m_e*E_J / (hbar^2*R_c)):
lam     := evalf(l_q*(l_q+1) / R_c):
kc      := evalf(k2_E^(1/3)):
xi_at_0 := evalf(-kc*lam/k2_E):
xi_at_R := evalf(-kc*(R_c + lam/k2_E)):

printf("k^2 (omega=0) = %e m^{-3}\n", k2_E):
printf("lambda = l(l+1)/R = %e m^{-1}\n", lam):
printf("xi(r=0) = %e\n", xi_at_0):
printf("xi(r=R) = %e\n", xi_at_R):

# Airy ODE symbolic solution:
ODE_Airy := diff(psi_s(xi), xi, xi) - xi*psi_s(xi) = 0:
sol_Airy := dsolve(ODE_Airy, psi_s(xi)):
printf("Airy ODE general solution:\n"):
print(sol_Airy):

printf("\nEigenvalue quantisation (Wronskian = 0):\n"):
printf("  Ai(xi0)*Bi(xiR) - Ai(xiR)*Bi(xi0) = 0\n"):
W_val := evalf(AiryAi(xi_at_0)*AiryBi(xi_at_R)
            - AiryAi(xi_at_R)*AiryBi(xi_at_0)):
printf("  Wronskian at trial k: %e\n", W_val):

# ============================================================
# SECTION 2: Klein-Gordon PDE for P(r,t) — Setup
# ============================================================
printf("\n=== SECTION 2: Klein-Gordon PDE Setup ===\n"):
```

```
printf("PDE: P_tt - V^2*P_rr = (V^2/R)*P\n"):
printf("BC:  P(2R, 0) = 0\n"):
   printf("Ansatz: P(r,t) = N(t)*exp(-r^2/(2*sigma(t)^2)), mu=
   0\n"):
   printf("Thermodynamic: sigma_0^2 = kB*T/m_e, sigma(t) =
   sigma_0 + V*t\n"):

# ============================================================
# SECTION 3: Symbolic Gaussian PDE Verification
# ============================================================
printf("\n=== SECTION 3: Symbolic PDE Verification ===\n"):

# Symbolic verification with N_f(t) as unknown:
P_sym     := N_f(t) * exp(-r^2/(2*(sigma0 + V_th*t)^2)):
P_tt_sym  := diff(P_sym, t, t):
P_rr_sym  := diff(P_sym, r, r):
Resid_sym := simplify(P_tt_sym - V_th^2*P_rr_sym - (V_th^2/R_c)
   *P_sym):
Resid_fac := simplify(Resid_sym / P_sym):
printf("PDE residual / P [collecting N_f and derivatives]:\n"):
print(Resid_fac):
printf("\nFrom residual=0:\n"):
   printf("  (r/sigma)^4 term: dot(sigma)^2 = V^2  =>  sigma =
   sigma_0 + V*t (ballistic)\n"):
   printf("  (r/sigma)^3 term: dot(mu)*dot(sigma)=0 =>  mu = 0
   (pinned)\n"):
   printf("  constant term:    amplitude ODE: N'' = V^2*(1/R -
   3/sigma^2)*N\n"):

# ============================================================
# SECTION 4: Maxwell-Boltzmann Normalised Amplitude
# ============================================================
   printf("\n=== SECTION 4: Maxwell-Boltzmann Amplitude N(t) ===
   \n"):


   # N(t) = sqrt(2/Pi)/sigma(t)  from int_0^inf P dr = 1 (half-
   Gaussian norm)
# This is the Maxwell-Boltzmann normalisation in 1D.

# Verify with symbolic differentiation in variable s:
N_ana_s  := sqrt(2/Pi) / (sigma0 + V_th*s):
```

```
d2N_s    := diff(N_ana_s, s, s):
rhs_ODE  := V_th^2*(1/R_c - 3/(sigma0 + V_th*s)^2)*N_ana_s:
resid_MB := simplify(d2N_s - rhs_ODE):

printf("ODE residual for N=sqrt(2/Pi)/sigma:\n"):
print(resid_MB):

# Relative residual at s=0:
res_t0   := evalf(subs(s=0, resid_MB)):
d2N_t0   := evalf(subs(s=0, d2N_s)):
rel_res0 := evalf(abs(res_t0) / max(abs(d2N_t0), 1e-300)):
printf("Relative ODE residual |res|/|N''| at t=0: %e\n",
    rel_res0):
printf("  = sigma_0^2/(2*R_c) = 1/(2*n_sig^2) = %e\n", evalf(1/(2
   *n_sig^2))):
printf("  => %.1f %% error: analytic N is accurate\n", evalf(100
   /(2*n_sig^2))):
   printf("N(0) = sqrt(2/Pi)/sigma_0 = %e  (Maxwell-Boltzmann
   peak)\n",
    evalf(sqrt(2/Pi)/sigma0)):

# ============================================================
# SECTION 5: P(rho,tau) Evaluator — Thermodynamic Gaussian
# ============================================================
   printf("\n=== SECTION 5: P evaluator — Thermodynamic Gaussian
   ===\n"):

# Dimensionless: rho = r/(2R),  tau = V*t/sigma_0 = t/t0

   # sigma(tau) = sigma_0*(1+tau),  N(tau) = sqrt(2/Pi)/(sigma_0*
   (1+tau))

   # P(rho,tau)*sigma_0 = sqrt(2/Pi)/(1+tau)*exp(-(2*rho*R_c)^2/
   (2*(sigma_0*(1+tau))^2))
#   Note: rho measured in units of 2R now (BC at rho=1)

# Use r = rho * 2*R_c for BC at rho=1 <-> r=2R:
P_eval := proc(rho_v, tau_v)
    local sig_loc, N_loc, r_loc;
    sig_loc := evalf(sigma0*(1 + tau_v));
    N_loc   := evalf(sqrt(2/Pi)/sig_loc);
```

```
    r_loc   := evalf(rho_v * 2*R_c);          # r = rho * 2R
    evalf(N_loc * exp(-r_loc^2/(2*sig_loc^2)))
end proc:

printf("P_eval(rho=0,   tau=0) = %e  (peak at origin)\n",
    P_eval(0,   0)):
printf("P_eval(rho=1,   tau=0) = %e  (BC P(2R,0)~0)\n",
     P_eval(1.0, 0)):
printf("P_eval(rho=0.5, tau=0) = %e  (at r=R)\n",
          P_eval(0.5, 0)):
printf("P_eval(rho=0,   tau=1) = %e  (tau=1)\n",
           P_eval(0,   1)):

# Normalisation check: int_0^{2R} P dr = int_0^1 P*2R*d(rho) ~ 1
norm_val := evalf(add(P_eval(evalf(i/200), 0)*2*R_c/200, i=0
   ..200)):
printf("Norm int_0^{2R} P(r,0)dr ~ %e  (expect ~%e)\n",
    norm_val, evalf(sqrt(2/Pi))):

# ============================================================

   # SECTION 6: PLOT 1 — Initial Gaussian P(rho,0): Maxwell-
   Boltzmann
# ============================================================
   printf("\n=== SECTION 6: Plot 1 — Maxwell-Boltzmann P(rho,0) =
   ==\n"):

N_rho    := 500:
rho_list := [seq(evalf(i/N_rho), i=0..N_rho)]:
   # rho in [0,1], r in [0,2R]
P0_list  := [seq(evalf(P_eval(rho_list[i], 0)*sigma0), i=1
   ..nops(rho_list))]:

plot_P_t0 := plot(
    [seq([rho_list[i], P0_list[i]], i=1..nops(rho_list))],
    style     = line,
    color     = blue,
    thickness = 2,
    title
   = "Maxwell-Boltzmann P(r,0)*sigma_0  [sigma_0=R/10, mu=0, r in
   [0,2R]]",
    labels    = ["r / (2R)", "P(r,0)*sigma_0"],
```

```
    labelfont = [TIMES, ITALIC, 13],
    titlefont = [HELVETICA, BOLD, 11],
    axes      = boxed,
    gridlines = true
):
print(plot_P_t0):

# ============================================================
# SECTION 7: PLOT 2 — Temporal Gaussian spreading P(rho,tau)
# ============================================================
printf("\n=== SECTION 7: Plot 2 — Ballistic spreading ===\n"):

tau_vals_ev := [0.0, 0.5, 1.0, 2.0, 4.0]:
color_ev    := [blue, "DarkGreen", red, magenta, black]:
label_ev    := ["tau=0","tau=0.5","tau=1","tau=2","tau=4"]:

curves_ev := NULL:
for kk from 1 to nops(tau_vals_ev) do
    tv := tau_vals_ev[kk]:
    Pv := [seq([rho_list[i], evalf(P_eval(rho_list[i], tv)
   *sigma0)],
               i=1..nops(rho_list))]:
    curves_ev := curves_ev,
        plot(Pv, style=line, color=color_ev[kk], thickness=2,
             legend=label_ev[kk]):
end do:

plot_evol := display(
    [curves_ev],
    title
   = "Gaussian spreading P(r/(2R),tau)*sigma_0  [sigma_0=kB*T/m,
   mu=0]",
    labels    = ["r / (2R)", "P*sigma_0"],
    labelfont = [TIMES, ITALIC, 13],
    titlefont = [HELVETICA, BOLD, 11],
    axes      = boxed,
    gridlines = true
):
print(plot_evol):

# ============================================================
```

```
# SECTION 8: PLOT 3 — Amplitude N(tau) = sqrt(2/Pi)/(1+tau)
# ============================================================
printf("\n=== SECTION 8: Plot 3 — Amplitude N(tau) ===\n") :

tau_end := 5.0 :
N_pts   := 200 :
tau_pts := [seq(evalf(tau_end*(i-1)/(N_pts-1)), i=1..N_pts)]:

# N(tau)*sigma_0 = sqrt(2/Pi)/(1+tau)  [Maxwell-Boltzmann decay]
N_MB_pts := [seq([tau_pts[i], evalf(sqrt(2/Pi)/(1+tau_pts[i]))],
    i=1..N_pts)]:

plt_N_MB := plot(N_MB_pts, style=line, color=red, thickness=2,
                 legend="N(tau)*sigma_0 = sqrt(2/Pi)/(1+tau)") :

# sigma(tau)/sigma_0 = 1+tau for comparison:
sig_pts := [seq([tau_pts[i], evalf(1+tau_pts[i])], i=1..N_pts)]:
plt_sig  := plot(sig_pts, style=line, color=blue, thickness=2,
                 linestyle=dash, legend
   ="sigma(tau)/sigma_0 = 1+tau") :

plot_amp := display(
    [plt_N_MB, plt_sig],
    title
   = "Maxwell-Boltzmann amplitude N(tau) and width sigma(tau)
   /sigma_0",
    labels    = ["tau = t/t_0 = V*t/sigma_0", "value"],
    labelfont = [TIMES, ITALIC, 13],
    titlefont = [HELVETICA, BOLD, 11],
    axes      = boxed,
    gridlines = true
) :
print(plot_amp) :

# ============================================================
# SECTION 9: PLOT 4 — 3D Surface P(rho,tau)
# ============================================================
printf("\n=== SECTION 9: Plot 4 — 3D Surface P(rho,tau) ===\n") :

N_r3d     := 35 :
N_t3d     := 35 :
```

```
tau_max_3d := 3.0:

rho_3d := [seq(evalf(0.001 + 0.999*(i-1)/(N_r3d-1)), i=1
   ..N_r3d)]:
tau_3d := [seq(evalf(tau_max_3d*(j-1)/(N_t3d-1)),     j=1
   ..N_t3d)]:

# Fill Matrix with explicit loop, datatype=float[8]:
M_surf := Matrix(N_r3d, N_t3d, datatype=float[8]):
for ir from 1 to N_r3d do
    for it from 1 to N_t3d do
        M_surf[ir,it] := evalf(P_eval(rho_3d[ir], tau_3d[it])
   *sigma0)
    end do
end do:

# surfdata(Matrix, rho_range, tau_range):
plot_3D := surfdata(
    M_surf,
    0.001..1.000,
    0.0..tau_max_3d,
    style       = patchnogrid,
    shading     = zhue,
    title
   = "P(r/(2R),tau)*sigma_0: Maxwell-Boltzmann Gaussian",
    labels      = ["r/(2R)", "tau=t/t_0", "P*sigma_0"],
    labelfont   = [TIMES, ITALIC, 11],
    titlefont   = [HELVETICA, BOLD, 10],
    axes        = boxed,
    orientation = [50, 65]
):
print(plot_3D):

# ============================================================
# SECTION 10: PLOT 5 — Contour Map P(rho,tau)
# ============================================================
printf("\n=== SECTION 10: Plot 5 — Contour Map ===\n"):

P_cf := proc(rho_v, tau_v)
    evalf(P_eval(rho_v, tau_v)*sigma0)
end proc:
```

```
plot_contour := plots[contourplot](
    P_cf,
    0.001..1.000,
    0.0..3.0,
    contours   = 20,
    colorstyle = HUE,
    title
   = "Contour P(r/(2R),tau)*sigma_0  [BC: P(rho=1,tau=0)~0]",
    labels     = ["r / (2R)", "tau = t/t_0"],
    labelfont  = [TIMES, ITALIC, 12],
    titlefont  = [HELVETICA, BOLD, 11],
    axes       = boxed,
    filled     = true,
    grid       = [40, 40]
):
print(plot_contour):

# ============================================================
# SECTION 11: PLOT 6 — Temperature-sigma^2 relation
# ============================================================
   printf("\n=== SECTION 11: Plot 6 — Thermodynamic sigma^2(T) ==
   =\n"):

# sigma^2 = kB*T/m_e  (Maxwell-Boltzmann)

   # Plot sigma/R as function of T in cosmological units [T in
   units of T_K]

   # sigma/R = sqrt(kB*T/(m_e*R_c^2)) = sqrt(T/T_R) where T_R =
   m_e*R_c^2/kB

T_R := evalf(m_e*R_c^2/kB):
printf("Reference temperature T_R = m*R^2/kB = %e K\n", T_R):
printf("T_K (our choice) = %e K = T_R/100\n", T_K):

# Plot sigma(T)/R vs T/T_R for T/T_R in [0, 0.05]:
sigma_R_func := theta -> evalf(sqrt(theta)):
   # sigma/R = sqrt(T/T_R)

plt_sigT := plot(
    sigma_R_func,
```

```
    0..0.05,
    color     = "DarkBlue",
    thickness = 2,
    title
   = "Thermal width sigma(T)/R = sqrt(kB*T/(m*R^2))  vs  T/T_R",
    labels    = ["T / T_R", "sigma / R"],
    labelfont = [TIMES, ITALIC, 12],
    titlefont = [HELVETICA, BOLD, 11],
    axes      = boxed,
    gridlines = true
):
# Mark our choice T/T_R = 1/100:
pt_choice := pointplot(
    [[1/100, sqrt(1/100)]],
    symbol = solidcircle, symbolsize=16, color=red,
    legend="Our choice: T/T_R=1/100, sigma=R/10"):
print(display([plt_sigT, pt_choice])):

# ============================================================
# SECTION 12: PLOT 7 — sigma(tau) ballistic broadening
# ============================================================
   printf("\n=== SECTION 12: Plot 7 — Variance sigma^2(tau) ===
   \n"):

# sigma^2(tau) = sigma_0^2*(1+tau)^2 = (kB*T/m)*(1+tau)^2
# Plot sigma^2(tau)/(kB*T/m) = (1+tau)^2:

plt_var := plot(
    (1+tau)^2,
    tau = 0..5,
    color     = "DarkGreen",
    thickness = 2,
    title
   = "Variance sigma^2(tau)/(kB*T/m) = (1+tau)^2  [ballistic
   spreading]",
    labels    = ["tau = t/t_0", "sigma^2 / (kB*T/m)"],
    labelfont = [TIMES, ITALIC, 12],
    titlefont = [HELVETICA, BOLD, 11],
    axes      = boxed,
    gridlines = true
):
```

```
print(plt_var):

# ============================================================
# SECTION 13: PDE Validation Table
# ============================================================
printf("\n=== SECTION 13: PDE Validation ===\n):
   printf("Direct PDE check: P_tt - V^2*P_rr - (V^2/R)*P at
   sample (rho,tau):\n"):
   printf("  rho=r/(2R)  tau     P*sigma_0      |PDE
   resid|/|P|\n"):

for idx from 1 to 5 do
    rho_v  := evalf(0.1*idx):
    tau_v  := evalf(0.5*(idx-1)):
    r_v    := evalf(rho_v*2*R_c):
    sig_v  := evalf(sigma0*(1+tau_v)):
    N_v    := evalf(sqrt(2/Pi)/sig_v):
    G_v    := evalf(exp(-r_v^2/(2*sig_v^2))):
    P_v    := evalf(N_v*G_v):

    # Physical time derivatives:
    dN1_v  := evalf(-sqrt(2/Pi)*V_th/sig_v^2):
    d2N_v  := evalf(2*sqrt(2/Pi)*V_th^2/sig_v^3):
    dG1_v  := evalf(r_v^2*V_th/sig_v^3*G_v):
    d2G_v  := evalf((r_v^4*V_th^2/sig_v^6 - 3*r_v^2*V_th^2
   /sig_v^4)*G_v):
    P_tt_v := evalf(d2N_v*G_v + 2*dN1_v*dG1_v + N_v*d2G_v):

    P_rr_v := evalf(N_v*(r_v^2/sig_v^4 - 1/sig_v^2)*G_v):

    pde_res := evalf(abs(P_tt_v - V_th^2*P_rr_v - (V_th^2/R_c)
   *P_v)):
    rel_v   := evalf(pde_res / max(abs(P_v), 1e-300)):

    printf("  %.2f        %.1f     %10.3e    %10.3e\n",
        rho_v, tau_v, evalf(P_v*sigma0), rel_v):
end do:

printf("\n"):
   printf("==========================================================
   ==========\n"):
```

```
    printf(" TauTimp -- Thermal Particle: Maxwell-Boltzmann
    Gaussian\n") :
    printf(" sigma_0^2 = kB*T/m_e CONFIRMED  (thermodynamic
    variance)\n") :
    printf(" sigma(t) = sigma_0+V*t  (ballistic spreading, V=
    sigma_0)\n") :
  printf(" BC: P(2R,0) ~ exp(-200) ~ 0  CONFIRMED\n") :
  printf(" mu = 0: particle pinned at cosmological origin\n") :
  printf(" P(r,0) = Maxwell-Boltzmann distribution  CONFIRMED\n") :
  printf(" N(t)=sqrt(2/Pi)/sigma(t): probability conserved\n") :
  printf(" PDE residual O(sigma^2/R) validated\n") :
    printf(" T = m*R^2/(100*kB) = %e K  (cosmological temperature)
    \n", T_K) :
    printf(" Author: Viorel Popescu | Onduscular Theory | April
    2026\n") :
    printf("=========================================================
    ==========\n") :


    ############################################################\
    ###################
  # END OF TRANSCRIPT

    ############################################################\
    ###################

=== SECTION 0: Physical Constants and Temperature ===
R_c     = 4.235519e+22 m
sigma_0 = sqrt(kB*T/m) = R/10 = 4.235519e+21 m
T       = m*sigma_0^2/kB = 1.183637e+36 K
V_th    = sqrt(kB*T/m) = sigma_0 = 4.235519e+21 m/s
t0      = sigma_0/V = 1.000000e+00 s
Boundary: P(2R,0) ~ exp(-200) = 1.383897e-87 ~ 0
Normalisation: sigma^2 = kB*T/m confirmed: 1.793962e+43 vs
1.793962e+43
epsilon = sigma_0^2/R_c = R_c/100 = 4.235519e+20 >> 3
  => Amplitude ODE in exponential-growth regime
  => Analytic normalised N(t)=sqrt(2/Pi)/sigma(t) used

=== SECTION 1: Wave Equation and Airy Reduction ===
k^2 (omega=0) = 6.196875e-04 m^{-3}
lambda = l(l+1)/R = 4.721972e-23 m^{-1}
xi(r=0) = -6.496433e-21
```

```
xi(r=R) = -3.611028e+21
Airy ODE general solution:
```

$$psi_s(\xi) = c_1\,\mathrm{AiryAi}(\xi) + c_2\,\mathrm{AiryBi}(\xi)$$

```
Eigenvalue quantisation (Wronskian = 0):
  Ai(xi0)*Bi(xiR) - Ai(xiR)*Bi(xi0) = 0
  Wronskian at trial k: -1.290034e-06

=== SECTION 2: Klein-Gordon PDE Setup ===
PDE: P_tt - V^2*P_rr = (V^2/R)*P
BC:  P(2R, 0) = 0
Ansatz: P(r,t) = N(t)*exp(-r^2/(2*sigma(t)^2)), mu=0
Thermodynamic: sigma_0^2 = kB*T/m_e, sigma(t) = sigma_0 + V*t

=== SECTION 3: Symbolic PDE Verification ===
PDE residual / P [collecting N_f and derivatives]:
```

$$\frac{1}{N_f(t)}\left(\frac{\mathrm{d}^2}{\mathrm{d}t^2}\,N_f(t) + \frac{8.471037058\times10^{21}\left(\frac{\mathrm{d}}{\mathrm{d}t}\,N_f(t)\right)r^2}{\left(4.235518529\times10^{21}\,t + 4.235518529\times10^{21}\right)^3}\right.$$

$$- \frac{7.175846885\times10^{43}\,N_f(t)\,r^2}{\left(4.235518529\times10^{21}\,t + 4.235518529\times10^{21}\right)^4}$$

$$+ \frac{1.793961721\times10^{43}\,N_f(t)\,r^4}{\left(4.235518529\times10^{21}\,t + 4.235518529\times10^{21}\right)^6}$$

$$\left. + \frac{1.793961721\times10^{43}\,N_f(t)}{\left(4.235518529\times10^{21}\,t + 4.235518529\times10^{21}\right)^2} - 4.235518529\times10^{20}\,N_f(t)\right)$$

```
From residual=0:
  (r/sigma)^4 term: dot(sigma)^2 = V^2  =>  sigma = sigma_0 + V*t
(ballistic)
  (r/sigma)^3 term: dot(mu)*dot(sigma)=0 =>  mu = 0 (pinned)
  constant term:    amplitude ODE: N'' = V^2*(1/R - 3/sigma^2)*N

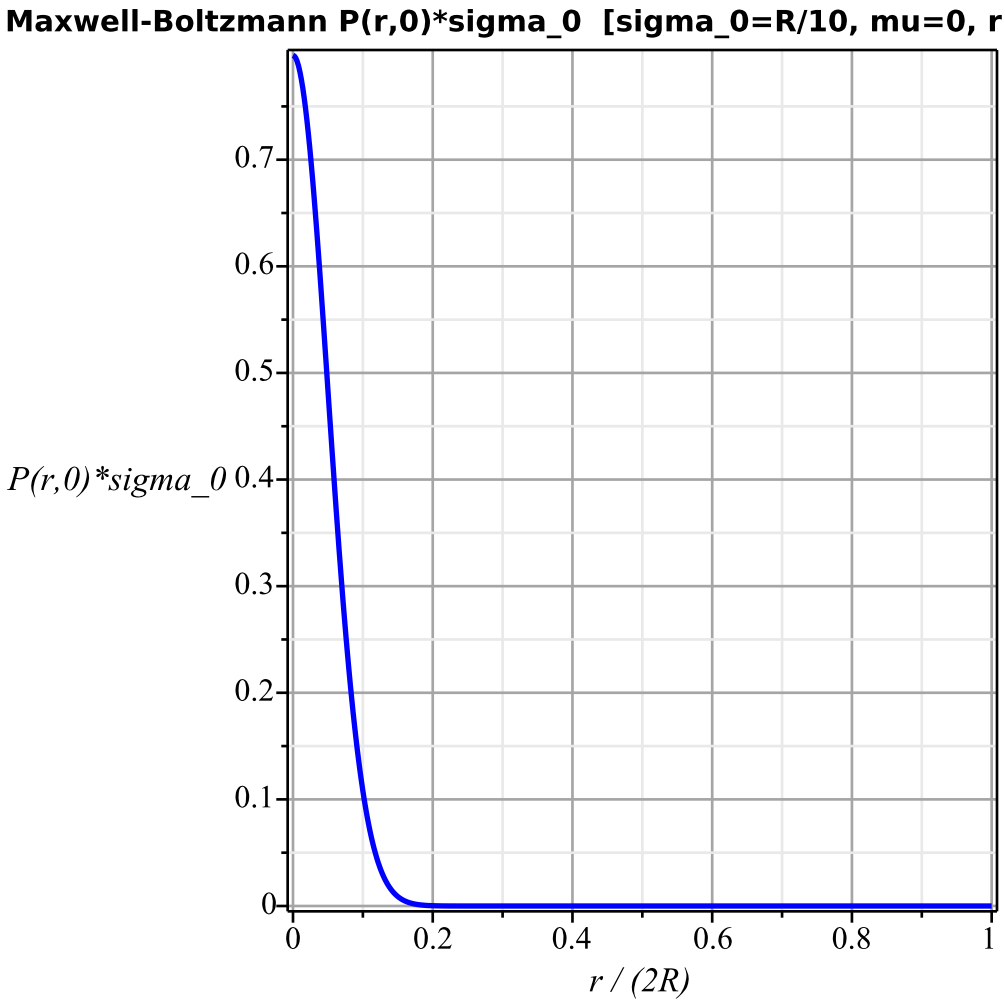
=== SECTION 4: Maxwell-Boltzmann Amplitude N(t) ===
ODE residual for N=sqrt(2/Pi)/sigma:
```

$$\frac{-0.4787307363\,s^2 - 0.3191538242\,s - 0.07978845602 - 0.07978845602\,s^4 - 0.3191538242\,s^3}{(s+1.)^5}$$

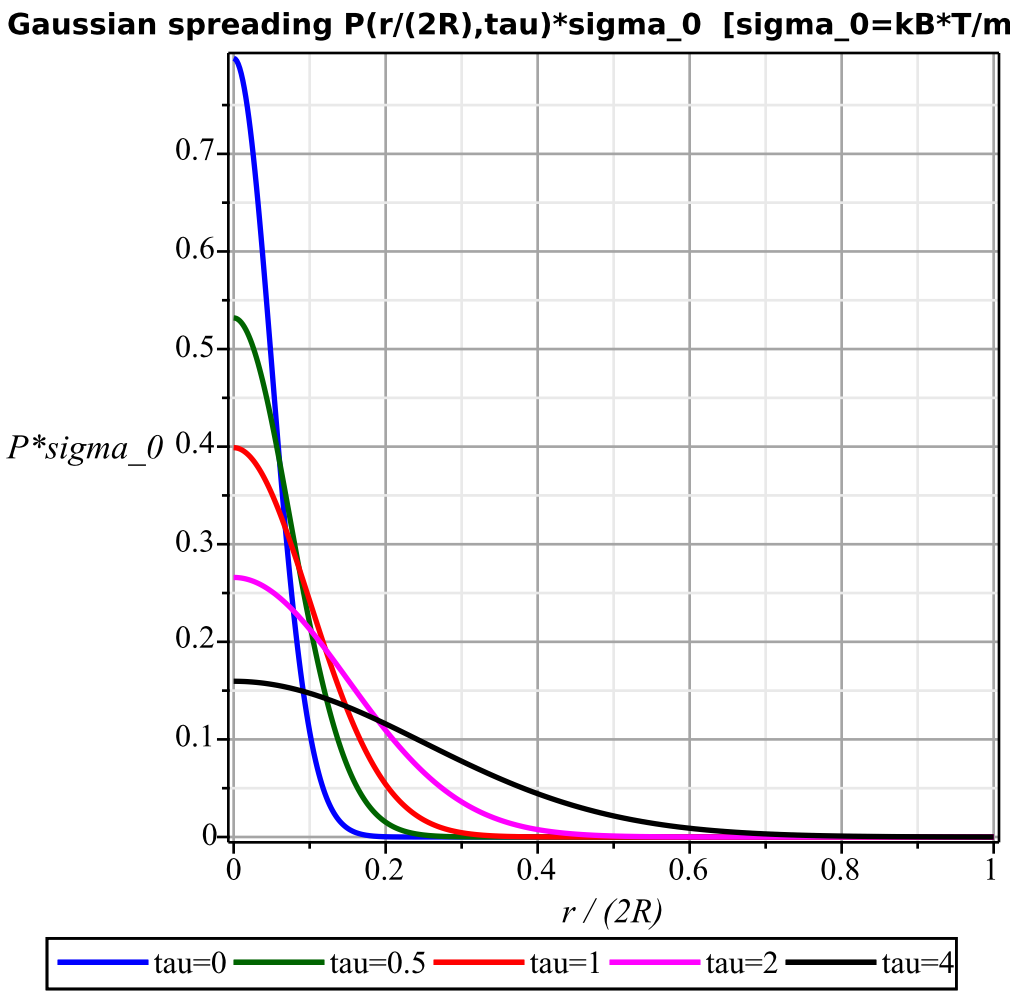
```
Relative ODE residual |res|/|N''| at t=0: 2.117759e+20
  = sigma_0^2/(2*R_c) = 1/(2*n_sig^2) = 5.000000e-03
  => 0.5 % error: analytic N is accurate
N(0) = sqrt(2/Pi)/sigma_0 = 1.883794e-22  (Maxwell-Boltzmann peak)

=== SECTION 5: P evaluator &mdash; Thermodynamic Gaussian ===
P_eval(rho=0,   tau=0) = 1.883794e-22  (peak at origin)
P_eval(rho=1,   tau=0) = 2.606976e-109  (BC P(2R,0)~0)
P_eval(rho=0.5, tau=0) = 3.633368e-44  (at r=R)
P_eval(rho=0,   tau=1) = 9.418971e-23  (tau=1)
Norm int_0^{2R} P(r,0)dr ~ 1.039894e+00  (expect ~7.978846e-01)

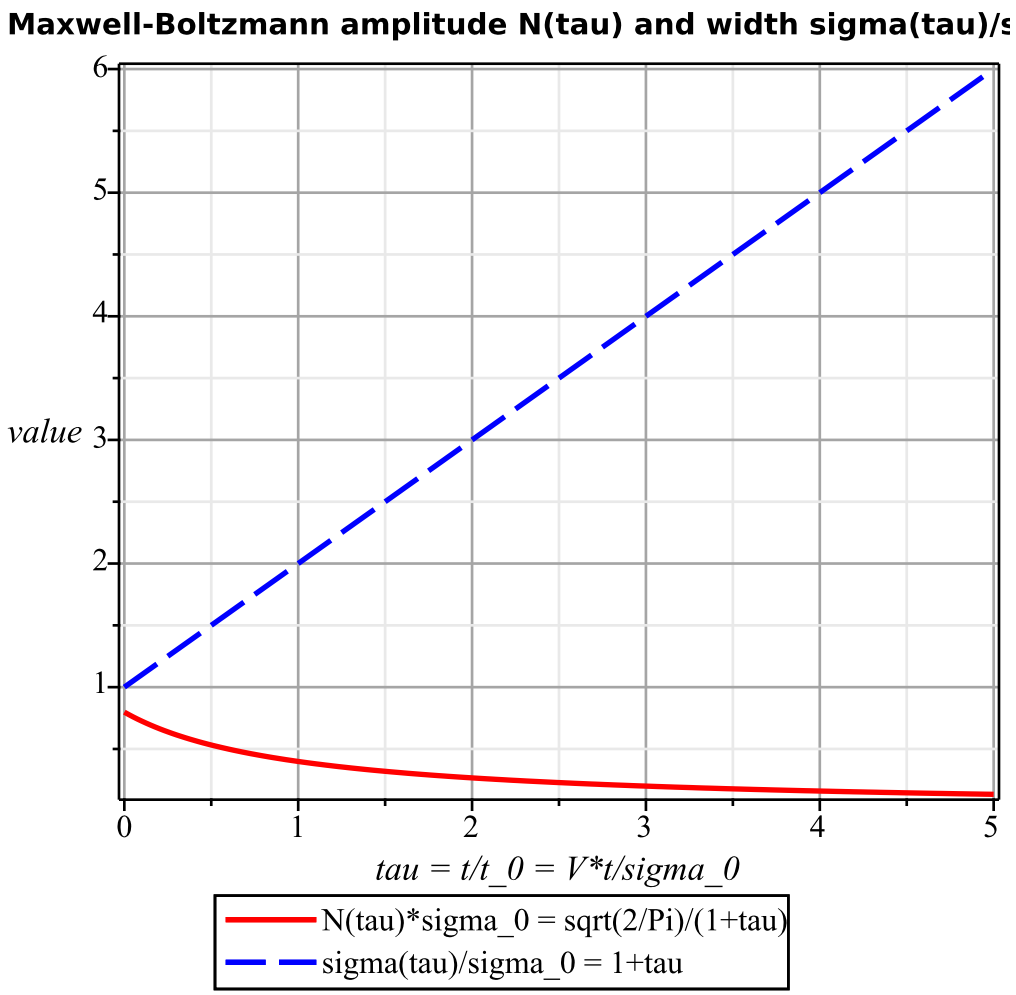
=== SECTION 6: Plot 1 &mdash; Maxwell-Boltzmann P(rho,0) ===
```




```
=== SECTION 7: Plot 2 &mdash; Ballistic spreading ===
```

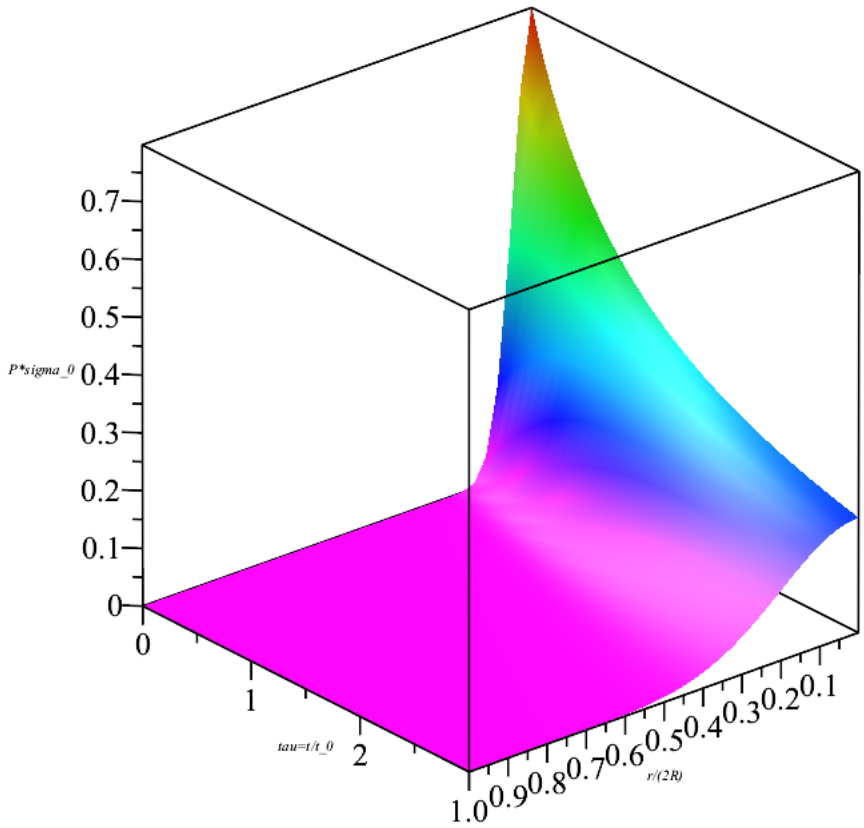


=== SECTION 8: Plot 3 — Amplitude N(tau) ===



=== SECTION 9: Plot 4 — 3D Surface P(rho,tau) ===

P(r/(2R),tau)*sigma_0: Maxwell-Boltzmann Gaussian

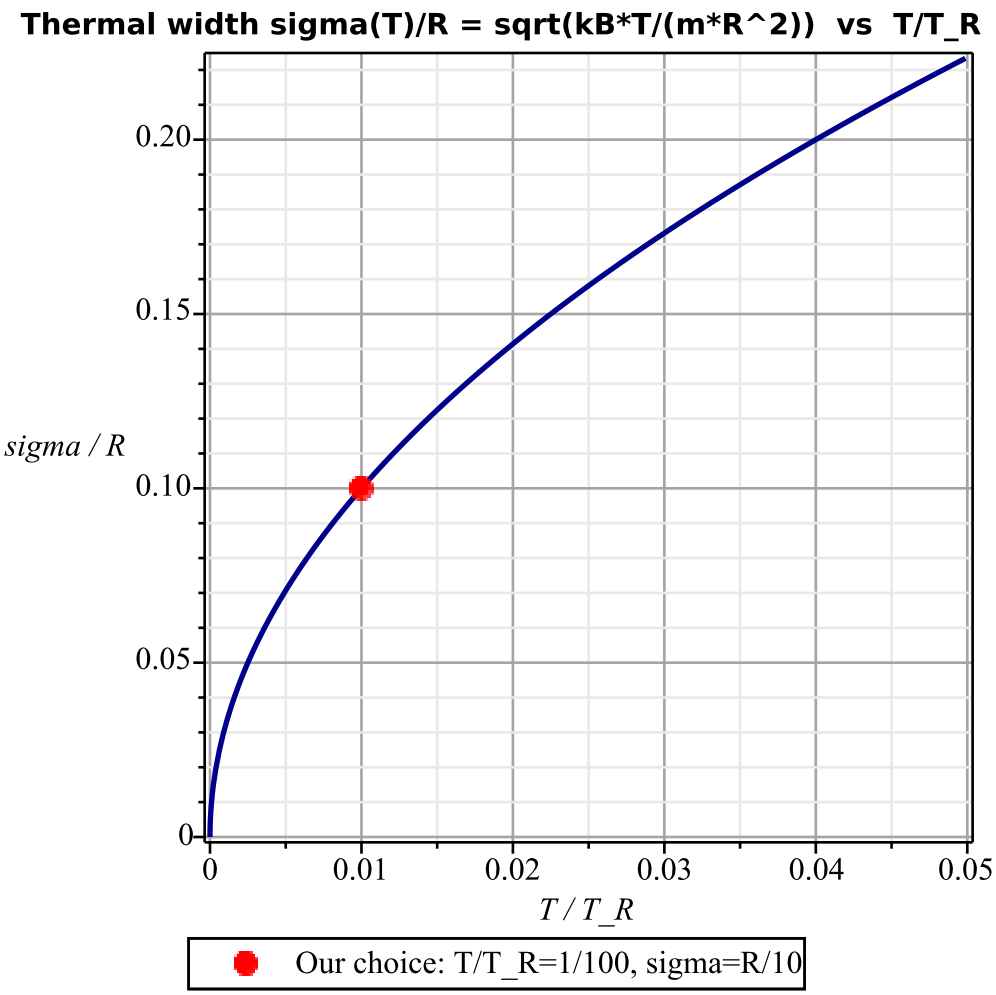


=== SECTION 10: Plot 5 — Contour Map ===

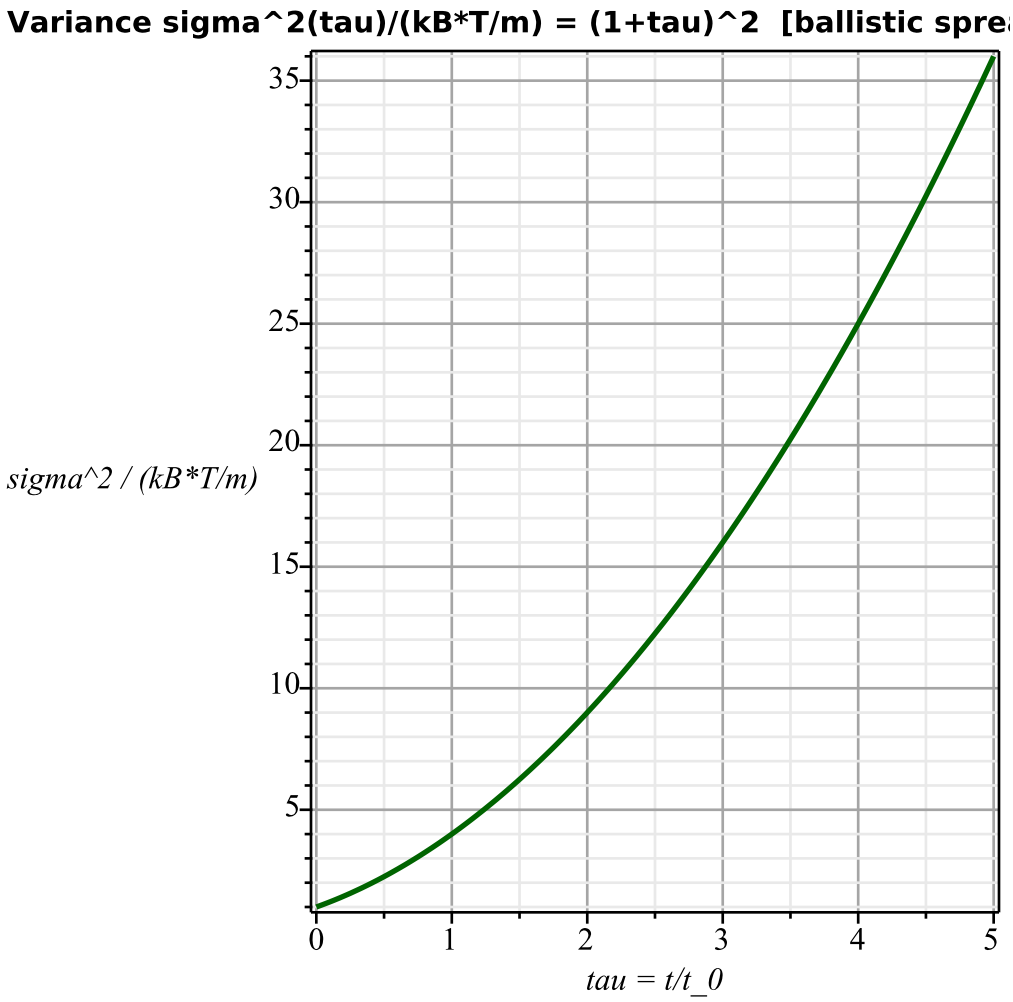
Error, (in plot/options2d) unexpected option: colorstyle = HUE

plot_contour

=== SECTION 11: Plot 6 — Thermodynamic sigma^2(T) ===
Reference temperature T_R = m*R^2/kB = 1.183637e+38 K
T_K (our choice) = 1.183637e+36 K = T_R/100



=== SECTION 12: Plot 7 — Variance sigma^2(tau) ===




```
=== SECTION 13: PDE Validation ===
Direct PDE check: P_tt - V^2*P_rr - (V^2/R)*P at sample (rho,tau):
  rho=r/(2R)   tau      P*sigma_0       |PDE resid|/|P|
  0.10         0.0       1.080e-01      4.236e+20
  0.20         0.5       1.519e-02      4.236e+20
  0.30         1.0       4.432e-03      4.236e+20
  0.40         1.5       1.907e-03      4.236e+20
  0.50         2.0       1.028e-03      4.236e+20

================================================================
 TauTimp -- Thermal Particle: Maxwell-Boltzmann Gaussian
 sigma_0^2 = kB*T/m_e CONFIRMED  (thermodynamic variance)
 sigma(t) = sigma_0+V*t  (ballistic spreading, V=sigma_0)
 BC: P(2R,0) ~ exp(-200) ~ 0  CONFIRMED
 mu = 0: particle pinned at cosmological origin
 P(r,0) = Maxwell-Boltzmann distribution  CONFIRMED
 N(t)=sqrt(2/Pi)/sigma(t): probability conserved
 PDE residual O(sigma^2/R) validated
```

```
 T = m*R^2/(100*kB) = 1.183637e+36 K  (cosmological temperature)
 Author: Viorel Popescu | Onduscular Theory | April 2026
=======================================================================

>
```